10 CUSTOM PIPES IN NESTJS

A custom pipe is a class YOU write that plugs into the exact same slot as ParseIntPipe or ValidationPipe (see theory/09-pipes.txt) — it runs BEFORE a route handler, and can transform and/or validate an argument. You reach for one when a built-in pipe doesn't cover a rule specific to your domain (e.g. "must be a positive integer," "this task ID must actually exist," "this string must be one of three allowed statuses").

THE PipeTransform INTERFACE

Every custom pipe implements PipeTransform, which requires exactly one method: `transform`.

```
import { PipeTransform, ArgumentMetadata } from '@nestjs/common';

export class MyPipe implements PipeTransform {
  transform(value: any, metadata: ArgumentMetadata) {
    // 1. do whatever check/conversion you need
    // 2. return the value that should reach the handler
    //    (converted, or unchanged)
    // 3. OR throw an exception to reject the request
    return value;
  }
}
```

value	The raw argument Nest is about to hand to your route handler (e.g. the ':id' string from the URL, or the parsed @Body() object).
metadata	Extra info ABOUT the argument: which kind it is ({ type: 'body' | 'query' | 'param' | 'custom' }), its declared TypeScript type (`metadata.metatype`), and the property name if relevant (`metadata.data` , e.g. 'id' for @Param('id', MyPipe)).

Whatever `transform` returns becomes the actual value your handler receives — same as the built-ins (a string '5' goes in, a number 5 comes out, if that's what your pipe does).

EXAMPLE 1 — SIMPLE VALIDATION/TRANSFORM (NO DEPENDENCIES)

A pipe that both converts AND validates in one step — reject anything that isn't a positive integer:

```
// positive-int.pipe.ts
import { PipeTransform, Injectable, BadRequestException } from '@nestjs/common';

@Injectable()
export class PositiveIntPipe implements PipeTransform {
  transform(value: string): number {
    const val = parseInt(value, 10);
    if (isNaN(val) || val <= 0) {
      throw new BadRequestException('Value must be a positive integer');
    }
    return val;
  }
}
```

```
// usage — parameter-level, exactly like ParseIntPipe
@Get(':id')
findOne(@Param('id', PositiveIntPipe) id: number) { ... }
```

Note `@Injectable()` on the pipe itself — this is what makes it eligible for Nest's DI container (see theory/07-dependency-injection.txt), which matters a lot for the next example.

EXAMPLE 2 — A CUSTOM PIPE THAT USES DEPENDENCY INJECTION

Because a pipe is just an `@Injectable()` class, it can have its own constructor-injected dependencies — including a service. This lets a pipe do something ParseIntPipe/ValidationPipe never could: check the database (or, here, in-memory store) BEFORE the handler even runs.

Applied to this project's task module — a pipe that takes the ':id' param, converts it to a number, AND fetches the actual Task, throwing 404 right there in the pipe if it doesn't exist:

```
// src/task/pipes/task-exists.pipe.ts
import {
  Injectable,
  PipeTransform,
  NotFoundException,
  BadRequestException,
} from '@nestjs/common';
import { TaskService } from '../task.service';
import type { Task } from '../interfaces/task.interface';

@Injectable()
export class TaskExistsPipe implements PipeTransform<string, Task> {
  constructor(private readonly taskService: TaskService) {}

  transform(value: string): Task {
    const id = parseInt(value, 10);
    if (isNaN(id)) {
      throw new BadRequestException('id must be a number');
    }
    return this.taskService.findOne(id); // throws NotFoundException itself if missing
  }
}

// task.controller.ts — usage
@Get(':id')
findOne(@Param('id', TaskExistsPipe) task: Task) {
  return task;               // already the full Task object — no
}                             // second lookup needed in the handler

@Delete(':id')
remove(@Param('id', TaskExistsPipe) task: Task) {
  return this.taskService.remove(task.id);
}
```

What changed vs the plain ParseIntPipe version:

- The handler receives the ALREADY-FETCHED Task object, not just a number — `findOne`'s "does it exist?" check now happens once, in the pipe, reusable across every route that needs "give me a real Task or 404."
- `PipeTransform<string, Task>` — the two generic params are INPUT type and OUTPUT type. Documents that this pipe takes a raw string and hands back a full Task.

- Because the pipe is `@Injectable()`, Nest resolves `TaskService` for it automatically, the exact same DI mechanism used everywhere else in this project — no `new TaskService()` needed.

EXAMPLE 3 — VALIDATING AGAINST A FIXED SET OF ALLOWED VALUES

```
// parse-status.pipe.ts
import { PipeTransform, Injectable, BadRequestException } from '@nestjs/common';

const ALLOWED = ['todo', 'in-progress', 'done'] as const;
type Status = (typeof ALLOWED)[number];

@Injectable()
export class ParseStatusPipe implements PipeTransform<string, Status> {
  transform(value: string): Status {
    if (!ALLOWED.includes(value as Status)) {
      throw new BadRequestException(
        `status must be one of: ${ALLOWED.join(', ')}`,
      );
    }
    return value as Status;
  }
}

@Get('by-status/:status')
findByStatus(@Param('status', ParseStatusPipe) status: Status) { ... }
```

(Nest also has a built-in ParseEnumPipe for this exact case if the values live in a real TS `enum` — a custom pipe like this is what you reach for when there's extra logic beyond a plain membership check.)

ASYNC CUSTOM PIPES

`transform` can be `async` and return a Promise — Nest awaits it before calling the handler. This is exactly what makes EXAMPLE 2 realistic for a true database lookup (`await this.taskRepository.findOneBy({ id })`) instead of the synchronous in-memory `findOne` used in this project.

```
async transform(value: string): Promise<Task> {
  const task = await this.taskService.findOneAsync(+value);
  if (!task) throw new NotFoundException();
  return task;
}
```

WHERE CUSTOM PIPES CAN BE APPLIED

Same four levels as any pipe (see theory/09-pipes.txt):

1. Parameter-level (most targeted): @Param('id', TaskExistsPipe) task: Task
2. Method-level: @UsePipes(TaskExistsPipe) @Get(':id')
3. Controller-level: @UsePipes(TaskExistsPipe) @Controller('task')
4. Global: app.useGlobalPipes(new TaskExistsPipe(...)); (awkward for pipes needing DI at the global level — usually only ValidationPipe is registered globally; injectable custom pipes are almost always applied at the parameter level instead.)

CUSTOM PIPE vs ValidationPipe — DIFFERENT JOBS

ValidationPipe	Generic — reads @IsString()/@IsNotEmpty() etc. decorators off ANY DTO class you give it. One pipe, reused for every DTO in the app (see theory/15-class-validator-and-class-transformer.txt).
Custom pipe	Specific — you write the exact rule/lookup logic yourself. Use it for checks that aren't "is this shape valid" but "does this ID exist," "is this within a fixed set," "convert this format" — things no generic decorator covers.

They compose fine together: a route can have `@Body() dto: CreateTaskDto` (validated by the global ValidationPipe) AND `@Param('id', TaskExistsPipe) task: Task` (validated by your custom pipe) on the same handler.

★ RULE OF THUMB

Reach for a custom pipe when the built-ins (ParseIntPipe, ParseUUIDPipe, ValidationPipe...) don't express the rule you need — especially "fetch-and-verify-existence" logic that would otherwise be copy-pasted as an `if (!found) throw new NotFoundException()` at the top of every handler. Mark it `@Injectable()` so it can use DI (a service, a repository) just like any other provider in the app.

> See also: theory/09-pipes.txt, theory/07-dependency-injection.txt, theory/15-class-validator-and-class-transformer.txt